UTEC
Universidad Tecnológica

Recorridos a nivel de árboles

Los recorridos de árboles son fundamentales para manipular y extraer información de estructuras de datos basadas en árboles. Existen varios tipos de recorridos, cada uno con su propia utilidad y características. A continuación, se presentan los recorridos más comunes: Preorden (preorder), en orden (inorder), posorden (postorder), recorrido en anchura (BFS) y recorrido en profundidad (DFS).

Recorridos en Profundidad (Depth-First Traversal)

Preorden (Preorder)

En el recorrido en preorden, se visita primero la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.

Algoritmo:
1. Visitar la raíz.
2. Recorrer el subárbol izquierdo en preorden.
3. Recorrer el subárbol derecho en preorden.

```
public void preorder(TreeNode node) {
    if (node == null) {
        return;
    }
    System.out.print(node.value + " ");
    preorder(node.left);
    preorder(node.right);
}
```

En Orden (Inorder)

En el recorrido en orden, se visita primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.

Algoritmo:
1. Recorrer el subárbol izquierdo en orden.
2. Visitar la raíz.
3. Recorrer el subárbol derecho en orden.

UTEC
Universidad Tecnológica

```java
public void inorder(TreeNode node) {
    if (node == null) {
        return;
    }
    inorder(node.left);
    System.out.print(node.value + " ");
    inorder(node.right);
}
```

Posorden (Postorder)

En el recorrido en posorden, se visita primero el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.

Algoritmo:
1. Recorrer el subárbol izquierdo en posorden.
2. Recorrer el subárbol derecho en posorden.
3. Visitar la raíz.

```java
public void postorder(TreeNode node) {
    if (node == null) {
        return;
    }
    postorder(node.left);
    postorder(node.right);
    System.out.print(node.value + " ");
}
```

Recorrido en Anchura (Breadth-First Search - BFS)

El recorrido en anchura, también conocido como búsqueda en anchura, visita todos los nodos a un nivel específico antes de pasar al siguiente nivel. Esto se logra utilizando una cola (queue).

Algoritmo:
1. Encolar la raíz.
2. Mientras la cola no esté vacía:
   - Desencolar un nodo.
   - Visitar el nodo.

- Encolar los nodos hijos (izquierdo y derecho) del nodo desencolado.

```
import java.util.LinkedList;
import java.util.Queue;

public void bfs(TreeNode root) {
    if (root == null) {
        return;
    }

    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);

    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.value + " ");

        if (node.left != null) {
            queue.add(node.left);
        }

        if (node.right != null) {
            queue.add(node.right);
        }
    }
}
```

Recorrido en Profundidad (Depth-First Search - DFS)

El recorrido en profundidad, también conocido como búsqueda en profundidad, puede ser implementado usando pilas (stack) de forma explícita o mediante recursión, como en los ejemplos de preorden, en orden y posorden.

Implementación con Pila

```
import java.util.Stack;

public void dfs(TreeNode root) {
    if (root == null) {
```

UTEC
Universidad Tecnológica

```
        return;
    }

    Stack<TreeNode> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        System.out.print(node.value + " ");

        if (node.right != null) {
            stack.push(node.right);
        }

        if (node.left != null) {
            stack.push(node.left);
        }
    }
}
```

Ejemplo Completo en Java

A continuación, un ejemplo completo de implementación de los recorridos en un árbol binario simple:

```
class TreeNode {
    int value;
    TreeNode left, right;

    TreeNode(int value) {
        this.value = value;
        left = right = null;
    }
}

public class ArbolRecorridos {
    public static void main(String[] args) {
        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(2);
```

UTEC
Universidad Tecnológica

```
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        root.left.right = new TreeNode(5);

        System.out.print("Preorden: ");
        preorder(root);
        System.out.println();

        System.out.print("En orden: ");
        inorder(root);
        System.out.println();

        System.out.print("Posorden: ");
        postorder(root);
        System.out.println();

        System.out.print("BFS: ");
        bfs(root);
        System.out.println();

        System.out.print("DFS: ");
        dfs(root);
        System.out.println();
    }

    public static void preorder(TreeNode node) {
        if (node == null) {
            return;
        }
        System.out.print(node.value + " ");
        preorder(node.left);
        preorder(node.right);
    }

    public static void inorder(TreeNode node) {
        if (node == null) {
            return;
        }
        inorder(node.left);
        System.out.print(node.value + " ");
        inorder(node.right);
    }
```

UTEC
Universidad Tecnológica

```java
    public static void postorder(TreeNode node) {
        if (node == null) {
            return;
        }
        postorder(node.left);
        postorder(node.right);
        System.out.print(node.value + " ");
    }

    public static void bfs(TreeNode root) {
        if (root == null) {
            return;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            System.out.print(node.value + " ");

            if (node.left != null) {
                queue.add(node.left);
            }

            if (node.right != null) {
                queue.add(node.right);
            }
        }
    }

    public static void dfs(TreeNode root) {
        if (root == null) {
            return;
        }

        Stack<TreeNode> stack = new Stack<>();
        stack.push(root);

        while (!stack.isEmpty()) {
            TreeNode node = stack.pop();
```

UTEC
Universidad Tecnológica

```
            System.out.print(node.value + " ");

            if (node.right != null) {
                stack.push(node.right);
            }

            if (node.left != null) {
                stack.push(node.left);
            }
        }
    }
}
```

Resumen

- Preorden (preorder): Visita la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.
- En orden (inorder): Visita el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.
- Posorden (postorder): Visita el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.
- Recorrido en anchura (BFS): Visita los nodos nivel por nivel utilizando una cola.
- Recorrido en profundidad (DFS): Visita los nodos en profundidad utilizando una pila (o recursión).